Product Specification: Admin Panel — Test Plan Creator

> **Product**: Galata Greek School CMS
> **Module**: Admin Panel → Test Plan Management
> **Version**: 1.0
> **Date**: 2026-03-08
> **Status**: Draft — Pending Review

---

1. Executive Summary

This specification defines a new **"Test Plan"** module within the existing Galata CMS Admin Panel. The module enables administrators to create, manage, and track structured test plans for verifying the quality of CMS-managed content (artifacts, events, venues, page content) before and after deployment.

The goal is to give the admin team a **built-in qupality assurance workflow** — no external tools required — that integrates directly with the existing Supabase-backed data model and the trilingual (TR/EN/EL) content architecture.

---

2. Problem Statement

Today, the Galata CMS has **7 content modules** (Artifacts, Events, Venue, History, İstanbul Rum, Categories, Page Content) but **no built-in mechanism** to:

- Define structured test cases for verifying content accuracy across languages
- Track whether published content has been reviewed after changes
- Create repeatable QA checklists before deploying content to production
- Record pass/fail results with evidence (screenshots, notes)

Content verification currently relies on ad-hoc manual checks, which can miss broken images, untranslated fields, or stale data.

---

3. Goals & Non-Goals

Goals
#	Goal
G1	Allow admins to create test plans with a title, description, and scope (which modules to test)
G2	Allow admins to add individual test cases to a plan, each with steps, expected results, and a target language
G3	Allow admins to execute test cases and record pass/fail/blocked results with notes

G4	Display a dashboard summary of test plan progress (% pass, fail, pending)
G5	Persist all test plan data in Supabase for history and auditability
G6	Follow the existing admin panel's design patterns (sidebar nav, form cards, trilingual tabs)

Non-Goals

#	Non-Goal
NG1	Automated test execution (this is a manual test tracking tool)
NG2	Integration with external test tools (Playwright, Jest, etc.)
NG3	Public-facing display of test plans — admin-only
NG4	User role/permission management beyond the existing admin auth

---

4. User Personas

Persona	Description	Primary Use
Content Administrator	Manages CMS content (artifacts, events, pages). Has `admin_users` credentials.	Creates test plans before publishing new content batches
QA Reviewer	Reviews published content for accuracy. Same admin credentials.	Executes test cases and records results

---

5. Feature Specification

5.1 New Sidebar Tab

- **Label**: `Test Planları` (Test Plans)
- **Icon**: `ClipboardList` from `lucide-react`
- **Position**: After "Sayfa İçerikleri" in the sidebar nav
- **Dashboard stat card**: Shows total test plans count

5.2 Test Plan List View

When the admin navigates to the Test Plans tab, they see:

Element	Description
Header	"Test Planları" with subtitle "Test planlarını oluşturun ve izleyin"
"+ Yeni Plan" button	Opens the create form
Table	Columns: `Plan Adı`, `Modül`, `Durum`, `İlerleme`, `Oluşturulma`, `İşlemler`
Status badges	`Taslak` (Draft), `Devam Ediyor` (In Progress), `Tamamlandı` (Completed)
Progress bar	Visual bar showing `X / Y` test cases passed

5.3 Test Plan Create/Edit Form

```
┌──────────────────────────────────────────────────┐
│  Yeni Test Planı                                 │
├──────────────────────────────────────────────────┤
│  Plan Adı:      [ _____________________________ ]│
│                                                  │
│  Açıklama:      [ _____________________________ ]│
│                 [ _____________________________ ]│
│                                                  │
│  Hedef Modül:   [ ▾ Arşiv Eserleri            ]  │
│                                                  │
│  Durum:         [ ▾ Taslak                    ]  │
├──────────────────────────────────────────────────┤
│              [ İptal ]    [ Planı Kaydet ]       │
└──────────────────────────────────────────────────┘
```

Fields:

Field	Type	Required	Description
`title`	`VARCHAR(255)`	☑	Test plan name
`description`	`TEXT`	☒	Free-text description of what this plan covers
`target_module`	`VARCHAR(50)`	☑	One of: `artifacts`, `events`, `venue`, `history`, `istanbul_rum`, `categories`, `content`, `all`
`status`	`VARCHAR(20)`	☑	`draft`, `in_progress`, `completed`

5.4 Test Case Management (within a Plan)

After saving a test plan, the admin can **add test cases** to it:

```
┌──────────────────────────────────────────────────────────────┐
│  Test Planı: "Batch 5 Artifact Yayını"     [ Düzenle ]       │
│  Modül: Arşiv Eserleri  │  Durum: Devam Ediyor               │
│  İlerleme: ████████░░  6/10                                  │
├──────────────────────────────────────────────────────────────┤
│  + Yeni Test Ekle                                            │
├──────┬──────────────────────────┬──────┬──────────┬──────────┤
│  #   │  Test Adı                │ Dil  │ Sonuç    │ İşlem    │
├──────┼──────────────────────────┼──────┼──────────┼──────────┤
│  1   │ Görsel yükleniyor mu?    │ ALL  │ ☑ Geçti  │ Düzenle  │
│  2   │ Başlık TR doğru mu?      │ TR   │ ☑ Geçti  │ Düzenle  │
│  3   │ Başlık EN doğru mu?      │ EN   │ ☑ Geçti  │ Düzenle  │
│  4   │ Başlık EL doğru mu?      │ EL   │ ☒ Kaldı  │ Düzenle  │
│  5   │ Künye bilgisi var mı?    │ ALL  │ ⏳ Bekle │ Çalıştır │
│ ...  │ ...                      │ ...  │ ...      │ ...      │
└──────┴──────────────────────────┴──────┴──────────┴──────────┘
```

Test Case Fields:

Field	Type	Required	Description
`title`	`VARCHAR(255)`	☑	Short description of what is being tested
`steps`	`TEXT`	❌	Step-by-step instructions for executing the test
`expected_result`	`TEXT`	☑	What the tester should observe if the test passes
`target_language`	`VARCHAR(5)`	☑	`tr`, [en](file:///c:/Users/1/Desktop/GalataGreekSchool-Page/GalataSchool/src/pages/AdminPage/components/AdminEvents.tsx#5-252), [el](file:///c:/Users/1/Desktop/GalataGreekSchool-Page/GalataSchool/src/pages/AdminPage/components/AdminArtifacts.tsx#102-109), or `all`
`result`	`VARCHAR(20)`	☑	`pending`, `pass`, `fail`, `blocked`
`notes`	`TEXT`	❌	Tester's notes, observations, or evidence
`executed_at`	`TIMESTAMPTZ`	❌	Auto-set when result changes from `pending`
`executed_by`	`VARCHAR(100)`	❌	Auto-set from current admin session

5.5 Test Execution Flow

1. Admin opens a test plan
2. Clicks **"Çalıştır"** (Execute) on a pending test case
3. A modal/inline panel expands showing:
   - Test title, steps, and expected result
   - Radio buttons: `☑ Geçti` (Pass) / `❌ Kaldı` (Fail) / `🚫 Engelli` (Blocked)
   - Textarea: Notes
4. Admin selects result, writes notes, clicks **"Sonucu Kaydet"**
5. Timestamp and admin email are auto-recorded
6. Progress bar updates in real-time

5.6 Dashboard Integration

On the main **Gösterge Paneli** (Dashboard):

- New stat card: **"Test Planı"** showing total count
- Color: **`indigo`** to distinguish from other modules
- Quick action: **"Yeni Test Planı"** → navigates to test plan creation

---

6. Database Schema

6.1 `test_plans` Table

```sql
CREATE TABLE public.test_plans (
```

```
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    target_module VARCHAR(50) NOT NULL DEFAULT 'all',
    status VARCHAR(20) NOT NULL DEFAULT 'draft'
        CHECK (status IN ('draft', 'in_progress', 'completed')),
    created_by VARCHAR(100),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

6.2 `test_cases` Table

```sql
CREATE TABLE public.test_cases (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    plan_id UUID NOT NULL REFERENCES public.test_plans(id) ON DELETE
CASCADE,
    title VARCHAR(255) NOT NULL,
    steps TEXT,
    expected_result TEXT NOT NULL,
    target_language VARCHAR(5) NOT NULL DEFAULT 'all'
        CHECK (target_language IN ('tr', 'en', 'el', 'all')),
    result VARCHAR(20) NOT NULL DEFAULT 'pending'
        CHECK (result IN ('pending', 'pass', 'fail', 'blocked')),
    notes TEXT,
    executed_at TIMESTAMPTZ,
    executed_by VARCHAR(100),
    sort_order INTEGER DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

6.3 RLS Policies

```sql
-- Allow all operations for authenticated admin sessions (matches
existing pattern)
ALTER TABLE public.test_plans ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.test_cases ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Allow all for anon" ON public.test_plans FOR ALL USING
(true);
CREATE POLICY "Allow all for anon" ON public.test_cases FOR ALL USING
(true);
```

---

7. Component Architecture

New Files

File	Purpose
`AdminTestPlans.tsx`	Main module component (list + form + detail views)

Modified Files

File	Change
[AdminPage.tsx](file:///c:/Users/1/Desktop/GalataGreekSchool-Page/GalataSchool/src/pages/AdminPage/AdminPage.tsx)	Add `test_plans` tab, stat card, quick action, import `AdminTestPlans`

Component Structure

```
AdminTestPlans
├── State: plans[], selectedPlan, testCases[], formData
├── Views:
│   ├── ListView (table of all plans)
│   ├── CreateForm (new/edit plan form)
│   └── DetailView (selected plan with test cases)
│       ├── TestCaseTable
│       ├── AddTestCaseForm
│       └── ExecuteTestCasePanel
└── CRUD Operations:
    ├── fetchPlans()
    ├── savePlan()
    ├── deletePlan()
    ├── fetchTestCases(planId)
    ├── saveTestCase()
    ├── deleteTestCase()
    └── executeTestCase(result, notes)
```

---

8. UI/UX Specifications

Design Principles
- **Consistency**: Reuse existing [AdminForms.css](file:///c:/Users/1/Desktop/GalataGreekSchool-Page/GalataSchool/src/pages/AdminPage/components/AdminForms.css) classes (`admin-form-card`, `admin-table`, `admin-btn-*`, `admin-badge`)
- **Module pattern**: Follow the same structure as [AdminArtifacts.tsx](file:///c:/Users/1/Desktop/GalataGreekSchool-Page/GalataSchool/src/pages/AdminPage/components/AdminArtifacts.tsx) (list/form toggle via `isEditing` state)
- **Status badges**: Use existing badge classes with new colors for test results

Result Status Colors

| Result | Badge Class | Color | Icon |

```
|--------|------------|-------|------|
| `pending` | `badge-pending` | Amber/Yellow | ⏳ |
| `pass` | `badge-success` | Green | ☑ |
| `fail` | `badge-danger` | Red | ✖ |
| `blocked` | `badge-blocked` | Gray | 🚫 |

### Progress Bar

```css
.test-progress-bar {
    height: 8px;
    border-radius: 4px;
    background: var(--admin-bg-secondary);
    overflow: hidden;
}
.test-progress-fill {
    height: 100%;
    border-radius: 4px;
    background: linear-gradient(90deg, #22c55e, #4ade80);
    transition: width 0.3s ease;
}
```

---

9. Pre-Built Test Plan Templates

To accelerate adoption, the module includes a **"Şablondan Oluştur"**
(Create from Template) option with these built-in templates:

Template	Module	Test Cases
Artifact QA Checklist	`artifacts`	Image loads, TR/EN/EL titles
present, Category assigned, Künye data complete, Status = published
Event QA Checklist	`events`	Cover image loads, Date valid,
TR/EN/EL titles, Thumbnail gallery, Detail page renders
Trilingual Content Audit	`all`	For each page: TR text
populated, EN text populated, EL text populated, No placeholder text
Archive Batch Review	`artifacts`	New items visible in L1 grid,
L2 detail renders, Breadcrumbs correct, Prev/Next navigation works

---

10. API Operations Summary

Operation	Supabase Call	Notes
List plans
`supabase.from('test_plans').select('*').order('created_at', { ascending:
false })` | |
| Get plan with case count | `supabase.from('test_plans').select('*,
test_cases(count)')` | Aggregate |
| Create plan | `supabase.from('test_plans').insert([payload])` | |
```

Update plan	`supabase.from('test_plans').update(payload).eq('id', id)`	
Delete plan	`supabase.from('test_plans').delete().eq('id', id)`	Cascades to test_cases
List test cases	`supabase.from('test_cases').select('*').eq('plan_id', id).order('sort_order')`	
Create test case	`supabase.from('test_cases').insert([payload])`	
Update test case result	`supabase.from('test_cases').update({ result, notes, executed_at, executed_by }).eq('id', id)`	Auto-set timestamps
Delete test case	`supabase.from('test_cases').delete().eq('id', id)`	

---

11. Success Metrics

Metric	Target
Admin can create a test plan in under **2 minutes**	☑
Admin can execute all test cases for a plan in a **single session** without page reload	☑
Progress bar accurately reflects real-time pass/fail/pending counts	☑
Template creation pre-populates **5+ test cases** instantly	☑
All data persists across sessions in Supabase	☑

---

12. Risks & Mitigations

Risk	Impact	Mitigation
Supabase RLS too permissive (anon access)	Low — admin panel is already behind login	Matches existing security pattern; can tighten later
Template test cases become outdated	Medium — new modules may not be covered	Templates are editable after creation
No screenshot/file attachment support for evidence	Low — notes field covers most cases	Can add file upload in v2

---

13. Future Enhancements (v2)

- [ ] Screenshot/file attachment for test evidence
- [ ] Test plan export to PDF/Markdown report
- [ ] Recurring test plan scheduling
- [ ] Slack/email notification on plan completion
- [ ] Auto-generate test cases from content diff (new artifacts added → auto-create verify cases)

---

14. Glossary

Term	Definition
Test Plan	A named collection of test cases targeting a specific module or content batch
Test Case	An individual check with steps, expected result, and a recorded outcome
Module	One of the 8 CMS content areas (artifacts, events, venue, etc.)
Template	A pre-built set of test cases that can be cloned into a new plan
Künye	Provenance/source metadata displayed in the archive L2 detail view